ESTUFA INTELIGENTE

Plataforma FIWARE · Projeto 2

Relatório técnico

Disciplina	CCM520 — Internet das Coisas
Integrante / RA	Lucas Belo Gaspardo — 22.224.007-9
API key / dispositivo	lbgestufa / estufa2604
Entidade	urn:ngsi-ld:Greenhouse:001 (Greenhouse)
Repositório GitHub	github.com/lucasbelo2604/projeto2-Lucas-lbgestufa
Data	13 de novembro de 2026

Escopo. Este relatório apresenta a arquitetura, as decisões de implementação e os resultados que puderam ser reproduzidos no ambiente. Os serviços FIWARE e os testes de provisionamento, MQTT, subscrições, persistência e painel foram executados. Não há ESP32 conectada nem dados de calibração do Projeto 1 neste ambiente.

1. Objetivo e arquitetura

O objetivo é representar a estufa como uma entidade de contexto NGSI-v2, mantendo a ESP32 no broker público HiveMQ e integrando medidas e comandos ao Orion por meio do IoT Agent JSON. As aplicações consultam o contexto no Orion; o histórico é persistido por Cygnus no MySQL e apresentado no Grafana, enquanto o consumidor recebe alertas de solo seco.

Origem → destino	Protocolo / porta	Conteúdo
ESP32 → HiveMQ público	MQTT TCP / 1883	Um JSON com sete medições/estados
HiveMQ ↔ ponte Mosquitto	MQTT bridge / 1883	Só tópicos da chave lbgestufa; medidas e resultado entram, comando sai
Mosquitto → IoT Agent	MQTT / 1883 interno	Tradução de object_id e ciclo de comando
IoT Agent → Orion	NGSI-v2 HTTP / 1026	Atualização de contexto e comandos
Orion → Cygnus	HTTP / 5050 interno	Notificações normalized dos sete atributos
Cygnus → MySQL	MySQL TCP / 3306 interno	Histórico em fei.estufa, uma linha por atributo
Grafana → MySQL	MySQL TCP / 3306 interno	Consultas com o usuário de leitura grafana

2. Identificadores e modelo de dados

A entidade usa o tipo Greenhouse, ID urn:ngsi-ld:Greenhouse:001, serviço fei, caminho /estufa, API key lbgestufa e device ID estufa2604. A ponte usa client ID ponte_lbgestufa; o firmware deriva esp32_lbgestufa_estufa2604. Os tópicos são /json/lbgestufa/estufa2604/attrs para medidas, /lbgestufa/estufa2604/cmd para comandos e /json/lbgestufa/estufa2604/cmdexe para respostas.

object_id	Atributo NGSI	Tipo
t	temperature	Number
rh	relativeHumidity	Number
sm	soilMoisture	Number
lx	illuminance	Number
r	relayState	Number
wa	windowAngle	Number
si	samplingInterval	Number

Atributos estáticos: name (Text) e location (geo:json Point). O valor [0.0, 0.0] no corpo Postman é uma sentinela provisória e precisa ser substituída pelas coordenadas reais antes do provisionamento. A calibração ADC do solo e do LDR também está provisória: os extremos configurados representam apenas a faixa nominal do ADC de 12 bits, não medições do circuito.

3. Decisões de implementação

Ponte filtrada. O IoT Agent JSON permanece ligado somente ao broker local. A ponte importa os tópicos de medidas e cmdexe e exporta o tópico cmd, todos limitados à API key e ao device ID do grupo. A direção separada reduz risco de loop e evita compartilhar o grupo de assinatura json do IoT Agent no broker público.

Provisionamento. O grupo rejeita dispositivos não provisionados (autoprovision=false). Os object_id t, rh, sm, lx, r, wa e si são usados de forma idêntica no firmware e no cadastro do dispositivo. Os quatro comandos têm validação de valor e resposta sob a mesma chave do comando.

Histórico e alerta. A subscrição do histórico envia somente as sete medições em normalized, formato que inclui estrutura/tipos para o Cygnus. A subscrição do alerta usa keyValues para enviar apenas soilMoisture, relayState e TimeInstant ao consumidor; o throttling de 10 s evita notificar a cada ciclo de 3 s. A condição deixa de ser verdadeira quando soilMoisture retorna a 30 ou mais.

Firmware. O ciclo de medidas usa ticks_ms e check_msg para continuar atendendo comandos mesmo quando setSending=0. Valores inválidos não atualizam atuadores. A resposta OK confirma que o dispositivo executou o comando; uma resposta ERRO indica que o dispositivo recebeu e recusou aquele valor.

4. Respostas R1–R8

R1 — Assinatura compartilhada e ponte

Uma shared subscription distribui cada mensagem correspondente entre os clientes participantes do mesmo grupo, em vez de entregar uma cópia a todos. Os IoT Agents JSON da turma usam o grupo fixo json e filtros curingas; ligados diretamente ao broker público, poderiam repartir uma medida entre agentes de grupos distintos. O agente que não conhece a chave/dispositivo descartaria a mensagem. A ponte mantém o IoT Agent isolado no Mosquitto local e encaminha apenas os tópicos do grupo. Medidas e cmdexe são in porque são publicadas pela ESP32 no broker público e precisam chegar ao agente local. O tópico cmd é out porque nasce no IoT Agent local e deve chegar à ESP32.

R2 — API key exclusiva

A chave compõe os tópicos e identifica o grupo no provisionamento. Se dois grupos usarem a mesma chave e device ID, ambos publicarão e assinarão os mesmos tópicos: medidas podem ser misturadas, comandos podem alcançar o dispositivo errado e o Orion pode receber contexto de outro grupo. Client IDs MQTT também precisam ser exclusivos, pois uma nova conexão com o mesmo ID desconecta a anterior.

R3 — Rejeitar dispositivos não provisionados

O broker público permite que qualquer cliente publique. Com autoprovisionamento, alguém poderia publicar um tópico inventado com a chave do grupo e criar entidades não autorizadas, poluindo o contexto e o histórico ou disparando alertas. autoprovision=false restringe o cadastro aos dispositivos explícitos.

R4 — Caminho de uma medida até o Grafana

A ESP32 envia um JSON numérico no tópico de attrs por MQTT TCP/1883 ao HiveMQ público. A ponte Mosquitto, conectada ao mesmo broker na porta 1883, importa somente esse tópico ao broker local. O IoT Agent assina o broker local em 1883, traduz object_id para atributos NGSI e atualiza o Orion por NGSI-v2 HTTP/1026. O Orion notifica o Cygnus por HTTP/5050 com normalized. O Cygnus grava cada atributo como uma linha em fei.estufa via MySQL/3306. O Grafana consulta MySQL pela rede Docker na porta 3306 e exibe as séries no painel; a interface do Grafana é publicada no host pela porta 3000.

R5 — Caminho do comando e significado de status

O Postman envia PATCH NGSI-v2 ao Orion solicitando setRelay=1. O Orion encaminha o comando ao provedor registrado do IoT Agent. O agente publica o JSON do comando no tópico cmd pelo Mosquitto local/1883; a ponte o exporta ao HiveMQ/1883 e a ESP32 inscrita executa a alteração no GPIO 26. A ESP32 publica {"setRelay":"OK"} ou {"setRelay":"ERRO: motivo"} em cmdexe; a ponte importa a resposta e o agente atualiza o atributo de comando no Orion. O status fica PENDING enquanto aguarda resposta e passa a OK quando o dispositivo respondeu. OK informa que houve resposta; não significa que o valor foi aceito. O texto da resposta está em setRelay_info.

R6 — Escuta de comandos com envio desativado

setSending=0 suspende apenas publicações periódicas. A ESP32 deve continuar processando check_msg e mantendo a assinatura MQTT, caso contrário não receberia setSending=1 e não poderia reativar o envio. Ao reativar, pode publicar uma medida imediatamente para atualizar o contexto.

R7 — Orion, MySQL e formatos de notificação

O Orion guarda o estado atual da entidade e os estados/respostas de comando. O MySQL conserva o histórico entregue pelo Cygnus, com linhas por atributo, horários, ID/tipo da entidade e valor textual. normalized inclui a estrutura do atributo e seus tipos/metadados, necessária à persistência do Cygnus. keyValues transmite valores simples e é adequado ao consumidor do alerta, que precisa somente de soilMoisture, relayState e TimeInstant.

R8 — Ameaças no broker público e mitigação

Dois ataques possíveis são (1) escutar medições e estados para inferir as condições da estufa e (2) publicar comandos ou medições falsos nos tópicos do grupo, alterando atuadores ou contexto. Em um sistema real, um broker privado/gerenciado com autenticação e ACL por dispositivo/tópico restringiria quem pode conectar e publicar/assinar. TLS protegeria confidencialidade e integridade do tráfego em trânsito. O IoT Agent e as APIs do Orion também precisariam autenticação/controle de acesso; TLS sozinho não impede um cliente autorizado de abusar dos tópicos a que tem acesso.

5. Estado de execução e pendências

O Docker Compose iniciou nove serviços, todos reportados como healthy. Orion, IoT Agent JSON, Cygnus, Grafana e o consumidor responderam; a fonte de dados Grafana conectou ao MySQL. A ponte HiveMQ↔Mosquitto foi exercitada nas duas direções e filtrou uma API key diferente. Grupo e dispositivo foram provisionados com HTTP 201; uma medida MQTT apareceu no Orion. A Etapa 1 do Newman passou com 6 requisições/10 asserções e a Etapa 2 com 6/31, sem falhas. Na Etapa 4, as duas subscrições foram criadas com HTTP 201. Após publicar uma amostra completa, as 11 verificações subsequentes passaram com 38 asserções, incluindo entrega ao Cygnus, persistência no MySQL, alerta e dashboard. A primeira checagem de entrega ocorreu antes de haver uma atualização posterior à criação da subscrição e, por isso, foi repetida após uma publicação; o resultado final registrado é o da verificação posterior. A Etapa 3 foi executada no projeto Wokwi personalizado, conectado ao HiveMQ e à plataforma local. Os 15 passos do Newman passaram sem falhas, incluindo relé, servo a 90°, intervalo de 5 s, rejeição de limites inválidos e pausa/retomada do envio. O projeto está em https://wokwi.com/projects/480526241555040257.

Escopo desta entrega: demonstração executada no Wokwi, com localização de demonstração [0,0] e faixas ADC configuradas para a simulação. Não houve calibração de sensores nem teste em placa física. Para uma instalação física futura, será necessário definir a localização e recalibrar os sensores.

Relatório preparado em 13/11/2026. Os testes do firmware e dos comandos foram realizados exclusivamente no simulador Wokwi; nenhum teste em hardware físico foi realizado. Os valores de localização e ADC são configurações de demonstração.